

TRACEFORGE V2 - BUILD DOCUMENTATION

Project: TraceForge v2 — Digital Forensics & Incident Response Toolkit

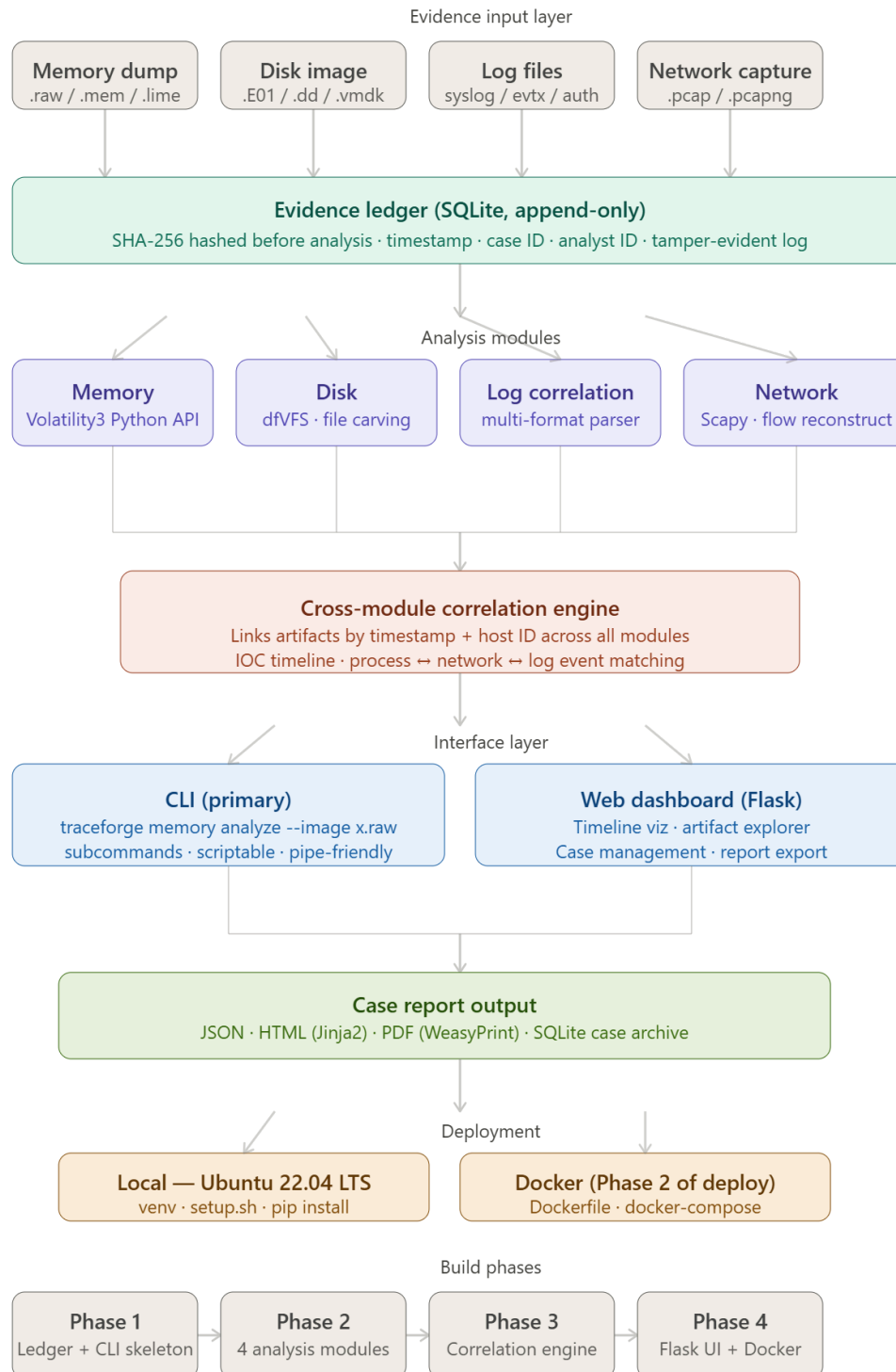
Version: 2.0

Target OS: Ubuntu 22.04 LTS

Stack: Python 3.11 · Volatility3 · dfVFS · Scapy · Flask · SQLite · Docker

How to use this document

Each phase has a gate condition at the top. **Do not start a phase until its gate passes.** This is the rule that v1 broke and why it never worked. Every step that produces code ends with a git commit. No exceptions.



Phase 0 — Environment & Project Scaffold

Status: Not Started **Gate:** Nothing in Phase 1 or beyond begins until every step in Phase 0 is verified and committed.

Estimated time: 1–2 hours on a clean Ubuntu 22.04 install.

Why Phase 0 exists

Every failure in v1 traced back to dependency problems discovered mid-build. pytsk3 failed to compile. Ubuntu 25.04 broke the Volatility3 dependency chain. Packages installed globally conflicted with the project. Phase 0 exists to eliminate all of that before a single line of application code is written.

When Phase 0 is done, you have a guaranteed working environment. Every subsequent phase builds on a stable base.

Step 0.1 — Install Ubuntu 22.04 LTS

Why this step comes first: The entire dependency chain — Volatility3, dfVFS, and their C-level libraries — is validated against Ubuntu 22.04 LTS. Using any other version (24.04, 25.04, Kali, Debian) introduces untested library versions and will cause silent failures in Phase 2.

What to do:

Install Ubuntu 22.04 LTS as one of the following:

- Bare metal (preferred for forensic work — no hypervisor overhead)
- VMware / VirtualBox VM (acceptable for development and testing)
- WSL2 on Windows (acceptable for development only — some memory forensics features require raw disk access not available in WSL2)

Verify the OS version before continuing:

```
lsb_release -a
```

Expected output:

```
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.x LTS
Release:      22.04
Codename:     jammy
```

Gate check: If the output does not say `22.04`, stop. Do not proceed on a different OS version.

Step 0.2 — Update the system

Why this comes before installing dependencies: Package lists must be current so `apt` resolves the correct versions of system libraries. Installing on a stale package index causes version conflicts that are extremely difficult to debug later.

```
sudo apt update && sudo apt upgrade -y
```

Wait for this to complete fully before the next step.

```
Setting up yelp (42.1-1ubuntu0.1) ...
Setting up libreoffice-gnome (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up gnome-shell (42.9-0ubuntu2.3) ...
Setting up libreoffice-impress (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up libreoffice-base-core (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up python3-uno (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up libreoffice-ogltrans (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up libreoffice-calc (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up libreoffice-writer (1:7.3.7-0ubuntu0.22.04.10) ...
Setting up update-manager (1:22.04.22) ...
Setting up gnome-shell-extension-ubuntu-dock (72~ubuntu5.22.04.3) ...
Setting up ubuntu-desktop-minimal (1.481.5) ...
Setting up ubuntu-desktop (1.481.5) ...
Processing triggers for desktop-file-utils (0.26-1ubuntu3) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1ubuntu3) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
Processing triggers for ufw (0.36.1-4ubuntu0.1) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for dbus (1.12.20-2ubuntu4.1) ...
Processing triggers for shared-mime-info (2.1-2) ...
Processing triggers for udev (249.11-0ubuntu3.20) ...
Processing triggers for install-info (6.8-4build1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for ca-certificates (20240203~22.04.1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
Processing triggers for initramfs-tools (0.140ubuntu13.5) ...
update-initramfs: Generating /boot/initrd.img-6.8.0-111-generic
theconsoler@ubuntu:~$
```

Step 0.3 — Install system-level dependencies

Why these are installed before the Python venv: These are C-level libraries that Python packages compile against during `pip install`. They must exist on

the system before `pip` tries to build packages like `dfvfs`. Installing them after the venv is created and packages have failed to build wastes significant time.

Install all dependencies in one command:

```
sudo apt install -y \  
  build-essential \  
  python3.11 \  
  python3.11-venv \  
  python3.11-dev \  
  python3-pip \  
  git \  
  tshark \  
  sleuthkit \  
  libewf-dev \  
  libvhdi-dev \  
  libvmdk-dev \  
  libbde-dev \  
  libsmdev-dev \  
  libffi-dev \  
  libssl-dev \  
  wget \  
  curl
```

```

theconsoler@ubuntu:~$ sudo apt install -y \
  build-essential \
  python3.11 \
  python3.11-venv \
  python3.11-dev \
  python3-pip \
  git \
  tshark \
  sleuthkit \
  libewf-dev \
  libvhdi-dev \
  libvmdk-dev \
  libbde-dev \
  libsmdev-dev \
  libffi-dev \
  libssl-dev \
  wget \
  curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.2-2ubuntu1.1).
wget set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu dpkg-dev fakeroot g++ g++-11 gcc gcc-11
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan6 libbcg729
  libc-ares2 libc-dev-bin libc-devtools libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0
  liberror-perl libewf2 libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-11-dev libitm
  libis-underscore liblsan0 liblua5.2-0 libnsl-dev libovthion3-dev libovthion3.10-dev libovthion

```



What each group provides:

Package(s)	Required by
<code>build-essential</code> , <code>python3.11-dev</code>	Compiling any Python package with C extensions
<code>python3.11</code> , <code>python3.11-venv</code>	The Python runtime for the project
<code>tshark</code>	Scapy's packet dissection
<code>sleuthkit</code>	dfVFS disk image parsing
<code>libewf-dev</code>	E01 / Expert Witness Format disk images
<code>libvhdi-dev</code>	VHD / VHDX disk images
<code>libvmdk-dev</code>	VMDK (VMware) disk images
<code>libbde-dev</code>	BitLocker encrypted volumes
<code>libsmdev-dev</code>	Physical storage media access
<code>libffi-dev</code> , <code>libssl-dev</code>	Python cryptographic packages

Verify tshark installed correctly (it requires a permissions choice):

```
tshark --version
```

If asked about non-superuser packet capture during install, select **Yes** to allow members of the `wireshark` group to capture packets. Then add your user:

```
sudo usermod -aG wireshark $USER
```

Log out and back in for the group change to take effect.

```
theconsoler@ubuntu:~$ tshark --version
TShark (Wireshark) 3.6.2 (Git v3.6.2 packaged as 3.6.2-2)

Copyright 1998-2022 Gerald Combs <gerald@wireshark.org> and contributors.
License GPLv2+: GNU GPL version 2 or later <https://www.gnu.org/licenses/gpl-2.0.html>
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compiled (64-bit) using GCC 11.2.0, with libpcap, with POSIX capabilities
(Linux), with libnl 3, with GLib 2.71.2, with zlib 1.2.11, with Lua 5.2.4, with
GnuTLS 3.7.3 and PKCS #11 support, with Gcrypt 1.9.4, with MIT Kerberos, with
MaxMind DB resolver, with nghttp2 1.43.0, with brotli, with LZ4, with Zstandard,
with Snappy, with libxml2 2.9.12, with libsmi 0.4.8.

Running on Linux 6.8.0-111-generic, with 11th Gen Intel(R) Core(TM) i5-11400H @
2.70GHz (with SSE4.2), with 7894 MB of physical memory, with GLib 2.72.4, with
zlib 1.2.11, with libpcap 1.10.1 (with TPACKET_V3), with c-ares 1.18.1, with
GnuTLS 3.7.3, with Gcrypt 1.9.4, with nghttp2 1.43.0, with brotli 1.0.9, with
LZ4 1.9.3, with Zstandard 1.4.8, with libsmi 0.4.8, with LC_TYPE=en_IN, binary
plugins supported (0 loaded).
theconsoler@ubuntu:~$ sudo usermod -aG wireshark $USER
theconsoler@ubuntu:~$
```

Step 0.4 — Create the project structure

Why the structure is fixed: The module loader, the CLI, and the correlation engine in Phase 3 all use relative imports that depend on this exact directory layout. Deviating from it now means refactoring imports later.

Create the full directory tree:

```
mkdir -p ~/traceforge/{traceforge/{modules,core},reports,samples,tests,docs,dashboard/templates}
cd ~/traceforge
```

Verify the structure looks exactly like this:

```

traceforge/           ← project root
├─ traceforge/        ← Python package
│   ├── __init__.py
│   ├── cli.py
│   └─ core/           ← ledger, hasher, artifact schema, correlator
│       ├── __init__.py
│       └─ modules/    ← memory, disk, logs, network
│           └─ __init__.py
├─ dashboard/         ← Flask app (built in Phase 4)
│   └─ templates/
├─ reports/           ← generated case reports go here
├─ samples/           ← test evidence files go here
├─ tests/             ← pytest test files
├─ docs/              ← this document and phase docs
└─ setup.py / pyproject.toml

```

```

theconsoler@ubuntu:~$ mkdir -p ~/traceforge/{traceforge/{modules,core},reports,samples,tests,docs,dashboard/templates}
theconsoler@ubuntu:~$ cd ~/traceforge
theconsoler@ubuntu:~/traceforge$ ls
dashboard docs reports samples tests traceforge
theconsoler@ubuntu:~/traceforge$

```

Create the `__init__.py` files:

```

touch ~/traceforge/traceforge/__init__.py
touch ~/traceforge/traceforge/core/__init__.py
touch ~/traceforge/traceforge/modules/__init__.py

```

```

theconsoler@ubuntu:~/traceforge$ touch ~/traceforge/traceforge/__init__.py
theconsoler@ubuntu:~/traceforge$ touch ~/traceforge/traceforge/core/__init__.py
theconsoler@ubuntu:~/traceforge$ touch ~/traceforge/traceforge/modules/__init__.py
theconsoler@ubuntu:~/traceforge$ ls
dashboard docs reports samples tests traceforge
theconsoler@ubuntu:~/traceforge$

```

Floppy Disk

Step 0.5 — Initialise git

Why git is initialised before the venv: The `.gitignore` must exist before the venv is created so the venv directory is never accidentally committed. A committed venv directory makes the repository unusable on any other machine.

Here's the complete working sequence from scratch:

1. Set git identity:

```

git config --global user.name "theconsoler"
git config --global user.email "your@email.com"

```


2. Store credentials so PAT is never asked again:

```
git config --global credential.helper store
```

3. Navigate to project and create `.gitignore`:

```
cd ~/traceforge
cat > .gitignore << 'EOF'
venv/
__pycache__/
*.pyc
*.pyo
*.pyd
.pytest_cache/
*.egg-info/
dist/
build/
reports/*.json
reports/*.html
reports/*.pdf
samples/
*.db
*.sqlite
.env
EOF
```

4. Activate venv and freeze requirements:

```
source venv/bin/activate
pip freeze > requirements.txt
```

5. Connect to GitHub repo:

```
git remote add origin https://github.com/theconsoler/TRACEFORGE-V2.git
```

6. Fix divergent branches:

```
git config pull.rebase false
git pull origin main --allow-unrelated-histories
```

If a text editor opens press `Ctrl+X` then `Enter`.

7. Stage and commit:

```
git add .
git commit -m "chore: phase 0 complete – project scaffold,
dependencies, gitignore"
```

8. Push:

```
git branch -M main
git push -u origin main --force
```

Username: `theconsoler`

Password: your PAT token (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)

Step 0.6 — Create and activate the virtual environment

Why a venv is mandatory: Installing packages globally on Ubuntu 22.04 breaks system Python in ways that are difficult to recover from. The venv isolates all project dependencies completely. Every time you work on TraceForge, you activate the venv first without exception.

```
cd ~/traceforge
python3.11 -m venv venv
source venv/bin/activate
```

Verify you are inside the venv:

```
which python
```

Expected output:

```
/home/<your-username>/traceforge/venv/bin/python
```

If this prints `/usr/bin/python` or similar, the venv is not active. Run `source venv/bin/activate` again.

Step 0.7 — Install core Python dependencies

Why this order matters: `volatility3` and `dfvfs` must be installed before packages that depend on them. Installing `pip` first ensures the package manager itself is current.

```
pip install --upgrade pip
```

Then install in this order:

```
pip install volatility3
pip install dfvfs
pip install scapy
pip install click rich loguru
pip install flask
pip install pandas
pip install jinja2 weasyprint
pip install pytest
```

```
(venv) theconsoler@ubuntu:~/traceforge$ deactivate
theconsoler@ubuntu:~/traceforge$ source venv/bin/activate
(venv) theconsoler@ubuntu:~/traceforge$ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.11/site-packages (26.1)
(venv) theconsoler@ubuntu:~/traceforge$ pip install volatility3
pip install dfvfs
pip install scapy
pip install click rich loguru
pip install flask
pip install pandas
pip install jinja2 weasyprint
pip install pytest
Requirement already satisfied: volatility3 in ./venv/lib/python3.11/site-packages (2.28.0)
Requirement already satisfied: pefile>=2024.8.26 in ./venv/lib/python3.11/site-packages (from volatility3) (2024.8.26)
Requirement already satisfied: dfvfs in ./venv/lib/python3.11/site-packages (20260411)
Collecting PyYAML>=3.10 (from dfvfs)
  Using cached pyyaml-6.0.3-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting dfdatetime>=20221112 (from dfvfs)
  Using cached dfdatetime-20260411-py3-none-any.whl
Collecting dtfabric>=20230518 (from dfvfs)
  Using cached dtfabric-20260411-py3-none-any.whl
Collecting libbde-python>=20220121 (from dfvfs)
  Using cached libbde_python-20240502-cp311-cp311-linux_x86_64.whl
Collecting libcaes-python>=20240114 (from dfvfs)
  Using cached libcaes_python-20251121.tar.gz (925 kB)
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Collecting libewf-python>=20131210 (from dfvfs)
```

Why not install everything in one line? If a package fails to build, a one-line install obscures which package caused the failure. Installing in stages gives you a clear error location.

Expected completion: Each package installs without errors. Warnings about deprecated features are acceptable. Errors are not — fix them before continuing.

Step 0.8 — Verify all critical imports

This step is mandatory. A dependency that appears installed but fails to import is a broken install. Finding this now takes 30 seconds. Finding it in Phase 2 mid-module-build costs an hour.

Run each verification individually:

```
python -c "import volatility3; print('volatility3 OK:', volatility3.__version__)"
python -c "import dfvfs; print('dfvfs OK')"
python -c "import scapy; print('scapy OK')"
python -c "import flask; print('flask OK:', flask.__version__)"
python -c "import click; print('click OK')"
python -c "import rich; print('rich OK')"
python -c "import loguru; print('loguru OK')"
python -c "import jinja2; print('jinja2 OK')"
python -c "import weasyprint; print('weasyprint OK')"
python -c "import pytest; print('pytest OK')"
```

Run all at once to get a final confirmation:

```
python -c "
import volatility3, dfvfs, scapy, flask, click, rich, loguru, jinja2, weasyprint, pytest
print('ALL IMPORTS OK — Phase 0 environment verified')
"
```

Expected output:

```
ALL IMPORTS OK – Phase 0 environment verified
```

If any import fails: Do not continue. Fix the failed import first. Common fixes:

- `dfvfs` import error → check `libewf-dev` and `libsmdev-dev` were installed in Step 0.3
- `weasyprint` import error → install missing system fonts: `sudo apt install -y libpango-1.0-0 libpangoft2-1.0-0`
- `scapy` import error → ensure `tshark` is installed and your user is in the `wireshark` group

Step 0.9 — Freeze requirements and make first commit

Why freeze before committing: `requirements.txt` captures exact package versions. Anyone who clones this repository can reproduce your exact environment. Without this, "works on my machine" becomes the default outcome.

```
pip freeze > requirements.txt
```

Make the first commit:

```
git add .  
git commit -m "chore: phase 0 complete – verified environment, project scaffold, dependencies"
```

Verify the commit:

```
git log --online
```

```

confg
credential.helper
curl
*.db
deactivate
dfvfs
dist/
dpkg
*.egg-info/
.env
--More--^C
(venv) theconsoler@ubuntu:~/traceforge$
(venv) theconsoler@ubuntu:~/traceforge$ python -c "
import volatility3, dfvfs, scapy, flask, click, rich, loguru, jinja2, weasyprint, pytest
print('ALL IMPORTS OK - Phase 0 environment verified')
"
ALL IMPORTS OK - Phase 0 environment verified
(venv) theconsoler@ubuntu:~/traceforge$ pip freeze > requirements.txt
(venv) theconsoler@ubuntu:~/traceforge$ git add .
(venv) theconsoler@ubuntu:~/traceforge$ git commit -m "chore: phase 0 complete - verified environment, project scaffold, dependencies"
[main 93035bd] chore: phase 0 complete - verified environment, project scaffold, dependencies
 1 file changed, 30 insertions(+)
(venv) theconsoler@ubuntu:~/traceforge$ git log --oneline
93035bd (HEAD -> main) chore: phase 0 complete - verified environment, project scaffold, dependencies
b9f0407 (origin/main) Merge branch 'main' of https://github.com/theconsoler/TRACEFORGE-V2
2273e4a chore: phase 0 complete - project scaffold, dependencies, gitignore
3dc59f9 Update README.md
1b1fcfb Initial commit
(venv) theconsoler@ubuntu:~/traceforge$

```

Expected output:

```
<hash> chore: phase 0 complete - verified environment, project scaffold, dependencies
```

Phase 0 completion checklist

Go through this list line by line before moving to Phase 1. Do not mark something complete if you are not 100% certain.

- ✓ `Ubuntu 22.04 LTS` confirmed (`lsb_release -a`)
- ✓ System packages installed without errors (Step 0.3)
- ✓ `tshark --version` runs successfully
- ✓ Project directory structure matches the tree in Step 0.4 exactly
- ✓ `.gitignore` created before `venv`
- ✓ Git repository initialised (`git init`)
- ✓ Virtual environment created with Python 3.11
- ✓ `which python` points to `venv/bin/python`
- ✓ All Python packages installed without build errors
- ✓ All import verifications pass — `ALL IMPORTS OK` printed
- ✓ `requirements.txt` generated
- ✓ First git commit made and visible in `git log`

When every box above is checked: Phase 0 is complete. Move to Phase 1.

What Phase 1 builds next

Once Phase 0 is complete, Phase 1 builds:

1. The SQLite evidence ledger with append-only chain-of-custody logging
2. The SHA-256 pre-analysis hasher
3. The standardised Artifact schema (the contract all modules must return)
4. The Click-based CLI skeleton with all subcommand stubs

None of Phase 1 is possible without Phase 0 complete. The import verifications in Phase 0 are the foundation that Phase 1's code depends on.

TraceForge v2 build documentation — Phase 0 of 4 Last updated: May 2026

[TRACEFORGE V2 — PHASE 1](#)

[TRACEFORGE v2 — PHASE 2](#)

[TRACEFORGE V2 — PHASE 3](#)

[TRACEFORGE V2 — PHASE 4](#)

[TRACEFORGE V2 — Interactive Launcher](#)

[COMPLETION AND FUTURE ENHANCEMENT IDEAS](#)

[TraceForge v2 — Complete Demo & Usage Guide](#)